

Use this cheat sheet for various SQL commands and their syntax, descriptions, and examples.

COUNT

COUNT function returns the number of rows that match a specified criterion.

Syntax (MySQL/DB2):

```
SELECT COUNT(column_name) FROM table_name WHERE condition;
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Example (MySQL/DB2):

```
SELECT COUNT(dep_id) FROM employees;
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



AVG

The *AVG* function returns the average value of a numeric column.

Syntax (MySQL/DB2):

```
SELECT AVG(column_name) FROM table_name WHERE condition;
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Example (MySQL/DB2):

```
SELECT AVG(salary) FROM employees;
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



SUM

The *SUM* function returns the total sum of a numeric column.

Syntax (MySQL/DB2):

```
SELECT SUM(column_name) FROM table_name WHERE condition;
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Example (MySQL/DB2):

```
SELECT SUM(salary) FROM employees;
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



MIN

The *MIN* function returns the smallest value of the selected column.

Syntax (MySQL/DB2):

```
SELECT MIN(column_name) FROM table_name WHERE condition;
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Example (MySQL/DB2):

```
SELECT MIN(salary) FROM employees;
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



MAX

The *MAX* function returns the largest value of the selected column.

Syntax (MySQL/DB2):

```
SELECT MAX(column_name) FROM table_name WHERE condition;
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Example (MySQL/DB2):

```
SELECT MAX(salary) FROM employees;
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



ROUND

The *ROUND* function rounds a number to a specified number of decimal places.

Syntax (MySQL/DB2):

```
SELECT ROUND(2number, decimals, operation) AS RoundValue;
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.

**Example (MySQL/DB2):**

```
SELECT ROUND(salary) FROM employees;
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



LENGTH

The *LENGTH* function returns the length of a string, in bytes.

Syntax (MySQL/DB2):

```
SELECT LENGTH(column_name) FROM table;
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.

**Example (MySQL/DB2):**

```
SELECT LENGTH(f_name) FROM employees;
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



UCASE

The *UCASE* function displays the column name in each table in uppercase.

Syntax (MySQL/DB2):

```
SELECT UCASE(column_name) FROM table;
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Example (MySQL/DB2):

```
SELECT UCASE(f_name) FROM employees;
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



LCASE

The *LCASE* function displays the column name in each table in lowercase.

Syntax (MySQL/DB2):

```
SELECT LCASE(column_name) FROM table;
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Example (MySQL/DB2):

```
SELECT LCASE(f_name) FROM employees;
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



DISTINCT

The *DISTINCT* function is used to display data without duplicates.

Syntax (MySQL/DB2):

```
SELECT DISTINCT column_name FROM table;
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Example (MySQL/DB2):

```
SELECT DISTINCT UCASE(f_name) FROM employees;
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



DAY

The *DAY* function returns the day of the month for a given date.

Syntax (MySQL/DB2):

```
SELECT DAY(column_name) FROM table
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Example (MySQL/DB2):

```
SELECT DAY(b_date) FROM employees where emp_id = 'E1002';
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



CURRENT_DATE

CURRENT_DATE is used to display the current date.

Syntax (MySQL/DB2):

```
SELECT CURRENT_DATE;
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Example (MySQL/DB2):

```
SELECT CURRENT_DATE;
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



DATEDIFF()

DATEDIFF() is used to calculate the difference between two dates or time stamps. The default value generated is the difference in number of days.

Syntax (MySQL/DB2):

```
SELECT DATEDIFF(date1, date2);
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Example (MySQL/DB2):

```
SELECT DATEDIFF(CURRENT_DATE, date_column) FROM table;
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



FROM_DAYS()

FROM_DAYS() is used to convert a given number of days to YYYY-MM-DD format.

Syntax (MySQL/DB2):

```
SELECT FROM_DAYS(number_of_days);
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Example (MySQL/DB2):

```
SELECT FROM_DAYS(DATEDIFF(CURRENT_DATE, date_column)) FROM table;
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



DATE_ADD()

DATE_ADD() is used to calculate the date after lapse of the mentioned number of units of date type. For example, if $n = 3$ and $\text{type} = \text{DAY}$, the result is a date 3 days after what is mentioned in the date column. The type variable can also be months or years.

Syntax (MySQL/DB2):

```
SELECT DATE_ADD(date, INTERVAL n type);
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Example (MySQL/DB2):

```
SELECT DATE_ADD(date, INTERVAL 3 DAY);
```

**Information:**

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



DATE_SUB()

DATE_SUB() is used to calculate the date prior to the record date by the mentioned number of units of date type. For example, if $n = 3$ and $\text{type} = \text{DAY}$, the result is a date 3 days before what is mentioned in the date column. The type variable can also be months or years.

Syntax (MySQL/DB2):

```
SELECT DATE_SUB(date, INTERVAL n type);
```

**Information:**

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.

**Example (MySQL/DB2):**

```
SELECT DATE_SUB(date, INTERVAL 3 DAY);
```

**Information:**

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Subquery

A *subquery* is a query within another SQL query and embedded within the WHERE clause. A subquery is used to return data that will be used in the main query as a condition to further restrict the data to be retrieved.

Syntax (MySQL/DB2):

```
SELECT column_name [, column_name ] FROM table1 [, table2 ] WHERE column_name OPERATOR (SELEC
T column_name [, column_name ] FROM table1 [, table2 ] [WHERE])
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Examples (MySQL/DB2):

```
Example 1:SELECT emp_id, f_name, l_name, salaryFROM employeeswhere salary< (SELECT AVG(salary
)FROM employees); Example 2:SELECT * FROM ( SELECT emp_id, f_name, l_name, dep_id FROM employ
ees) AS emp4all; Example 3:SELECT * FROM employees WHERE job_id IN (SELECT job_id FROM job
s);
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Implicit Inner Join

An *implicit inner join* combines two or more records but displays only matching values in both tables. Inner join applies only the specified columns.

Syntax (MySQL/DB2):

```
SELECT column_name(s) FROM table1, table2 WHERE table1.column_name = table2.column_name;
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Example (MySQL/DB2):

```
SELECT * FROM employees, jobs where employees.job_id = jobs.job_id;
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Implicit Cross Join

An *implicit cross join* is defined as a Cartesian product where the number of rows in the first table is multiplied by the number of rows in the second table.

Syntax (MySQL/DB2):

```
SELECT column_name(s) FROM table1, table2;
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Example (MySQL/DB2):

```
SELECT * FROM employees, jobs;
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.

